

1- Ejercicio Estadísticas Descriptivas Básicas

Enunciado:

Dado un conjunto de datos que representa las edades de un grupo de personas, calculamos las siguientes medidas estadísticas:

1. **Media**
2. **Mediana**
3. **Desviación estándar**
4. **Mínimo y máximo**

Mostramos los resultados en la consola.

Breve reseña:

Este ejercicio permite aprender a **resumir un conjunto de datos con medidas estadísticas básicas**.

- La **media** muestra el valor promedio.
- La **mediana** indica el valor central, útil cuando hay valores extremos.
- La **desviación estándar** refleja la **variabilidad** de los datos.
- El **mínimo y máximo** muestran el **rango** de los datos.

Este tipo de análisis es la base para cualquier estudio exploratorio de datos, ayudando a entender **distribución y dispersión** antes de realizar gráficos o modelos más avanzados.

En R (RStudio)

```
# Datos de ejemplo: edades de un grupo de personas
```

```
edades <- c(23, 25, 30, 22, 28, 27, 35, 24, 29, 31)
```

```
# Calcular estadísticas descriptivas básicas
```

```
media <- mean(edades)
```

```
mediana <- median(edades)
```

```
desv_est <- sd(edades)
```

```
minimo <- min(edades)
```

```
maximo <- max(edades)
```

```
# Mostrar resultados
```

```
cat("Media:", media, "\n")
```

```
cat("Mediana:", mediana, "\n")
```

```
cat("Desviación estándar:", desv_est, "\n")
```

```
cat("Mínimo:", minimo, "\n")
```

```
cat("Máximo:", maximo, "\n")
```

Preguntas:

1. ¿Qué diferencia hay entre la media y la mediana?

- En una distribución **simétrica como la normal**, **media $\approx$ mediana**.
- Si la distribución estuviera **sesgada** (con valores muy extremos hacia un lado), **media y mediana serían distintas**, y esa diferencia indica asimetría en los datos.

2. ¿Qué nos indica la desviación estándar sobre los datos?

- La **desviación estándar (σ)** mide **cuánto se dispersan los datos** alrededor de la media.
- En tu simulación:
 - scale=10 significa **$\sigma = 10$ cm**,
 - Entonces, **aprox. 68% de las alturas** estarán entre **160 y 180 cm** (media $\pm 1\sigma$).

Interpretación:

- **σ pequeña** → datos concentrados alrededor de la media (baja variabilidad).
- **σ grande** → datos muy dispersos, más diferencias de altura entre personas.

En Python (VS Code o Colab)

```
# Datos de ejemplo: edades de un grupo de personas
```

```
edades = [23, 25, 30, 22, 28, 27, 35, 24, 29, 31]
```

```
import numpy as np
```

```
media = np.mean(edades)
```

```
mediana = np.median(edades)
```

```
desv_est = np.std(edades)
```

```
minimo = np.min(edades)
```

```
maximo = np.max(edades)
```

```
print(f"Media: {media}")
```

```
print(f"Mediana: {mediana}")
```

```
print(f"Desviación estándar: {desv_est}")
```

```
print(f"Mínimo: {minimo}")
```

```
print(f"Máximo: {maximo}")
```

2- Ejercicio de Visualización de datos con histograma

Enunciado:

1. Generamos 1,000 datos aleatorios que sigan una **distribución normal estándar** (media = 0, desviación estándar = 1).
2. Dibujamos un **histograma** de los datos simulados.
3. Superponemos la **curva de densidad teórica** de la distribución normal sobre el histograma.

Breve

reseña:

Este ejercicio permite **visualizar una distribución normal** y comparar datos simulados con su modelo teórico.

- El **histograma** muestra cómo se distribuyen los datos aleatorios.
- La **curva roja de densidad** indica cómo debería verse una distribución normal perfecta.

Es útil para entender **conceptos de probabilidad, distribuciones continuas** y para **verificar si los datos siguen un patrón esperado**.

En R (RStudio)

```
library(ggplot2)
```

```
set.seed(42)
```

```
datos <- rnorm(500, mean = 50, sd = 10)
```

```
ggplot(data.frame(x = datos), aes(x)) +
```

```
  geom_histogram(bins = 30, fill = "skyblue", color = "black") +
```

```
  ggtitle("Histograma de datos simulados") +
```

```
  theme_minimal()
```

Preguntas:

3- ¿Cómo es la forma del histograma?

El histograma tiene **forma de campana (Gaussiana)**.

Es **simétrico**, con la mayor concentración de datos alrededor de la media y menos frecuencias a medida que nos alejamos hacia los extremos.
Esta forma refleja el comportamiento típico de una **distribución normal**.

4- ¿Los datos parecen distribuidos normalmente? ¿Por qué?

Sí, parecen distribuidos normalmente porque:

El **histograma es simétrico** respecto a la media.

La **frecuencia de datos** disminuye progresivamente hacia ambos extremos.

La **curva de densidad teórica (roja)** se ajusta bastante bien al histograma.

En resumen, la **simetría** y la **forma de campana** son indicadores visuales claros de que los datos siguen un patrón normal.

En Python (VS Code o Colab)

```
import matplotlib.pyplot as plt
import numpy as np
np.random.seed(42)
datos = np.random.normal(loc=50, scale=10, size=500)
plt.hist(datos, bins=30, color='skyblue', edgecolor='black')
plt.title('Histograma de datos simulados')
plt.show()
```

3- Ejercicio Teorema Central del Límite (TCL) — Simulación

Enunciado:

1. Generamos muchas muestras aleatorias de números uniformes (entre 0 y 1).
2. Calculamos la **media de cada muestra** y guardamos en un vector.
3. Dibujamos un histograma de las medias muestrales.
4. Observamos cómo se aproxima a una **distribución normal**, ilustrando el **Teorema Central del Límite**.

Breve

reseña:

Este ejercicio muestra **por qué la distribución normal es tan importante en estadística**:

- Incluso si los datos originales provienen de una **distribución uniforme** (plana),
- **Las medias de muchas muestras** tienden a seguir una **distribución normal**.

Esto es el **Teorema Central del Límite (TCL)**, base para **intervalos de confianza** y **pruebas estadísticas** en análisis de datos.

En R (RStudio)

```
library(ggplot2)

n <- 30

repeticiones <- 1000

set.seed(123)

medias_muestras <- replicate(repeticiones, mean(runif(n)))

ggplot(data.frame(x = medias_muestras), aes(x)) +
  geom_histogram(aes(y = after_stat(density)), bins = 30, fill =
"lightgreen", color = "black") +
  stat_function(fun = dnorm, args = list(mean = 0.5, sd =
1/sqrt(12*n)), color = "red", linewidth = 1) +
  ggtitle("Distribución de medias muestrales (TCL)") +
  theme_minimal()
```

Preguntas:

4- ¿Qué forma tiene la distribución de las medias muestrales?

La distribución tiene **forma de campana (aproximadamente normal)**.

Es **simétrica** alrededor de la media ≈ 0.5 (porque el promedio de números uniformes entre 0 y 1 es 0.5).

A medida que aumentás el número de repeticiones o el tamaño de la muestra n , la forma se hace **más suave y más parecida a una normal**.

5- ¿Por qué la distribución se parece a una normal?

Según el **Teorema Central del Límite (TCL)**:

La distribución de las **medias de muchas muestras** tiende a una **normal**, sin importar la forma de la distribución original.

En este caso:

1. Los datos originales son **uniformes** (distribución plana).

2. Al tomar muchas muestras y calcular sus medias, el **promedio suaviza la variabilidad**.
3. El resultado es una **distribución de medias** con forma de campana, centrada en 0.5.

Conclusión:

Esto demuestra por qué la **normal** aparece tan seguido en estadística y por qué el TCL es la base de **intervalos de confianza y pruebas de hipótesis**.

En Python

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

n = 30
repeticiones = 1000
np.random.seed(123)
medias_muestras = [np.mean(np.random.uniform(size=n)) for _ in
range(repeticiones)]

plt.hist(medias_muestras, bins=30, density=True, color='lightgreen',
edgecolor='black')

# Parámetros de la normal teórica
mu = 0.5
sigma = 1 / np.sqrt(12 * n)
x = np.linspace(min(medias_muestras), max(medias_muestras),
100)
plt.plot(x, norm.pdf(x, mu, sigma), 'r-', lw=2)
plt.title('Distribución de medias muestrales (TCL)')
plt.show()
```

OTROS EJERCICIOS

1- Probabilidad y lanzamiento de dados

a. **Enunciado:**

Simulamos 1,000 lanzamientos de un dado de seis caras. Calculamos la frecuencia de cada número y visualizamos los resultados en un gráfico de barras.

b. **Breve**

reseña:

Este ejercicio introduce la **simulación de eventos aleatorios** y la **ley de los grandes números**. Permite entender que al aumentar la cantidad de lanzamientos, las frecuencias observadas tienden a acercarse a la probabilidad teórica de $1/6$ para cada cara. Es útil para comprender **probabilidad empírica** y la relación entre **simulación y teoría**.

○ **R**

```
set.seed(101)
# Simulamos 1000 lanzamientos de un dado
dados <- sample(1:6, size = 1000, replace = TRUE)
# Calculamos la frecuencia de cada cara
tabla_dados <- table(dados)
tabla_dados
# Visualización
barplot(tabla_dados, main = "Frecuencia de
lanzamientos de dado",
        col = rainbow(6), xlab = "Cara del dado", ylab =
"Frecuencia")
```

○ **Python**

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(101)
# Simulamos 1000 lanzamientos de un dado
dados = np.random.randint(1, 7, size=1000)
# Contamos frecuencias
valores, frecuencias = np.unique(dados,
return_counts=True)
print(dict(zip(valores, frecuencias)))
# Visualización
plt.bar(valores, frecuencias,
color=plt.cm.rainbow(np.linspace(0,1,6)))
```

```
plt.title("Frecuencia de lanzamientos de dado")  
plt.xlabel("Cara del dado")  
plt.ylabel("Frecuencia")  
plt.show()
```

- Preguntas:

- ¿Las frecuencias son iguales o hay variaciones?

No son exactamente iguales.

En 1,000 lanzamientos, es normal que haya **pequeñas diferencias** entre las frecuencias de cada cara debido al azar.

Ejemplo hipotético de frecuencias:

1: 170
2: 162
3: 168
4: 167
5: 160
6: 173

Todas están **cerca de $1,000/6 \approx 167$** , pero no exactamente iguales.

- Si aumentamos a 100,000 lanzamientos, ¿qué esperarías que ocurra?

Las frecuencias relativas se acercarán más a $1/6 \approx 0.1667$ para cada cara.

Aunque siempre habrá **variaciones aleatorias**, con más lanzamientos la **proporción** de cada cara se vuelve más estable.

Esto ilustra la **Ley de los Grandes Números**, que dice que la **frecuencia empírica** tiende a la **probabilidad teórica** al aumentar el número de ensayos.

2- Correlación y diagrama de dispersión

a. **Enunciado:**

Generamos un conjunto de datos con dos variables relacionadas. Dibujamos un diagrama de dispersión y calculamos la correlación entre las variables.

b. **Breve**

reseña:

Este ejercicio permite analizar **relaciones entre variables**. Un diagrama de dispersión muestra visualmente si existe **relación lineal positiva, negativa o nula**, mientras que la **correlación** cuantifica esta relación. Es un paso inicial para aprender sobre **análisis bivariado** y sirve como base para **regresión**.

- **R**
set.seed(202)
x <- rnorm(50, mean = 50, sd = 10)
y <- x * 0.5 + rnorm(50, mean = 0, sd = 5)
plot(x, y, main = "Diagrama de dispersión X vs Y",
 xlab = "X", ylab = "Y", pch = 19, col = "blue")
cor(x, y)

- **Python**
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(202)
x = np.random.normal(50, 10, 50)
y = 0.5 * x + np.random.normal(0, 5, 50)
plt.scatter(x, y, color='blue')
plt.title("Diagrama de dispersión X vs Y")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
print("Correlación:", np.corrcoef(x, y)[0,1])

- Preguntas:

- ¿El gráfico muestra una tendencia lineal positiva o negativa?

Positiva.

- A medida que **X aumenta**, **Y también tiende a aumentar**.
 - Los puntos siguen aproximadamente una **línea ascendente**, aunque con algo de dispersión.
 - Si la correlación es cercana a 0.5, ¿qué significa?

Indica una **correlación positiva moderada**.

Hay tendencia a que **Y aumente con X**, pero con **variabilidad considerable**.

No es una relación perfecta, por lo que otros factores influyen en Y.

3- Regresión lineal simple

a. **Enunciado:**

Simulamos una variable X y una variable Y que siga aproximadamente una relación lineal. Ajustamos un modelo de regresión lineal simple y dibujamos la recta de regresión sobre los puntos.

b. **Breve**

reseña:

Este ejercicio enseña a **ajustar modelos lineales** y a **interpretar la pendiente e intercepto**. La regresión lineal es una de las herramientas más utilizadas para **predecir valores** y **entender relaciones lineales** entre variables numéricas. Además, permite reforzar el concepto de **tendencia central de los datos**.

- **R**

```
set.seed(303)
x <- 1:20
y <- 2*x + 5 + rnorm(20, sd = 3)
modelo <- lm(y ~ x)
summary(modelo)
plot(x, y, pch=19, col="darkgreen")
abline(modelo, col="red", lwd=2)
```

- **Python**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
np.random.seed(303)
x = np.arange(1, 21).reshape(-1, 1)
y = 2*x.flatten() + 5 + np.random.normal(0, 3, 20)
modelo = LinearRegression().fit(x, y)
print("Pendiente:", modelo.coef_[0])
print("Intercepto:", modelo.intercept_)
plt.scatter(x, y, color='green')
plt.plot(x, modelo.predict(x), color='red', linewidth=2)
plt.title("Regresión lineal simple")
plt.show()
```

- **Preguntas:**

- ¿El modelo captura bien la relación entre X e Y ?

Sí, si la dispersión de los puntos alrededor de la recta roja es baja.

El modelo capta la **tendencia lineal principal**, aunque los puntos no caen exactamente sobre la recta porque hay **ruido aleatorio**.

Esto se puede cuantificar con el **R^2 del modelo** (cercano a 1 = ajuste fuerte, cercano a 0 = ajuste débil).

- ¿Qué representan la pendiente y el intercepto?

Pendiente (coeficiente de X):

Indica **cuánto aumenta Y por cada unidad que aumenta X**.

Intercepto (β_0):

Es el **valor estimado de Y cuando $X = 0$** .

Juntos, definen la **recta de regresión** que mejor se ajusta a los datos según el método de mínimos cuadrados.

4- Clasificación con K-NN

a. **Enunciado:**

Generamos un conjunto de puntos en dos grupos (clases A y B). Representamos en un gráfico de dispersión y luego, usando el algoritmo k-NN, clasificamos un nuevo punto (3.5, 3.5) en una de las clases.

b. **Breve**

reseña:

Este ejercicio introduce un **algoritmo de clasificación supervisada** muy simple: **k-Nearest Neighbors, o vecinos más próximos (k-NN)**. Enseña cómo **clasificar nuevos datos según su cercanía a datos conocidos**, mostrando la importancia de la **distancia entre puntos** en el análisis. Es un primer paso hacia la **inteligencia artificial y machine learning**.

○ **R**

```
library(class)
# Creamos un dataset simple
set.seed(404)
x1 <- c(rnorm(10, 2), rnorm(10, 5))
x2 <- c(rnorm(10, 2), rnorm(10, 5))
clase <- factor(c(rep("A",10), rep("B",10)))
datos <- data.frame(x1, x2, clase)
plot(x1, x2, col=clase, pch=19, main="Datos para k-NN")

# Nuevo punto para clasificar
nuevo <- data.frame(x1=3.5, x2=3.5)
pred <- knn(train=datos[,1:2], test=nuevo, cl=clase, k=3)
pred
```

- **Python**

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
np.random.seed(404)
x1 = np.concatenate([np.random.normal(2,1,10),
np.random.normal(5,1,10)])
x2 = np.concatenate([np.random.normal(2,1,10),
np.random.normal(5,1,10)])
clase = np.array(['A']*10 + ['B']*10)
X = np.column_stack((x1,x2))
plt.scatter(x1, x2, c=(clase=='B'), cmap='bwr', s=80)
plt.title("Datos para k-NN")
plt.show()
modelo = KNeighborsClassifier(n_neighbors=3)
modelo.fit(X, clase)
nuevo = np.array([[3.5,3.5]])
print("Clase predicha:", modelo.predict(nuevo))
```

- **Preguntas:**

- ¿Qué clase obtiene el nuevo punto (3.5,3.5)?

Probablemente **clase A**, porque el punto (3.5, 3.5) suele estar **más cerca del grupo A** en esta simulación.

La **clase real la devuelve pred en R o modelo.predict en Python**, y puede variar levemente según la simulación aleatoria.

- ¿Qué pasaría si cambiamos $k=1$ o $k=5$?

- **$k = 1$:**

- Clasificación basada **solo en el vecino más cercano**.
 - **Muy sensible al ruido** o puntos atípicos.

- **$k = 5$:**

- Clasificación según la **mayoría de 5 vecinos más cercanos**.
 - **Más estable**, pero suaviza la frontera de decisión y puede asignar un punto al grupo mayoritario cercano.

5- Serie Temporal y promedio móvil

- a. **Enunciado:**

Simulamos datos de ventas mensuales durante 2 años (24 meses).
Graficamos la serie temporal y calculamos un **promedio móvil de 3 meses** para suavizar la serie.

b. **Breve****reseña:**

Este ejercicio enseña los conceptos básicos de **series temporales** y **suavizado de datos**. El **promedio móvil** es una técnica inicial para **detectar tendencias** en presencia de variaciones aleatorias. Es útil para análisis de **ventas, producción, tráfico web** y cualquier indicador que evolucione con el tiempo.

• **R**

```
set.seed(505)
ventas <- ts(rnorm(24, mean = 200, sd = 20), frequency = 12)
```

```
plot(ventas, type="o", col="blue", main="Serie de Ventas Mensuales", ylab="Ventas")
```

```
# Promedio móvil de 3 meses
```

```
promedio_movil <- filter(ventas, rep(1/3, 3), sides = 2)
lines(promedio_movil, col="red", lwd=2)
```

• **Python**

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
np.random.seed(505)
ventas = pd.Series(np.random.normal(200, 20, 24))
plt.plot(ventas, marker='o', color='blue', label='Ventas')
plt.plot(ventas.rolling(window=3, center=True).mean(), color='red',
linewidth=2, label='Promedio móvil 3 meses')
plt.title("Serie de Ventas Mensuales")
plt.legend()
plt.show()
```

• **Preguntas:**

- ¿Qué hace el promedio móvil sobre la serie original?
 - **Suaviza la serie temporal**, reduciendo la variabilidad aleatoria o ruido presente en los datos.
 - Ayuda a visualizar la tendencia subyacente sin distracciones por fluctuaciones cortas.
- ¿Qué utilidad tiene para detectar tendencias?
 - Permite identificar **tendencias generales o patrones** a lo largo del tiempo.
 - Facilita la toma de decisiones al mostrar el comportamiento promedio y evitar reaccionar a cambios pasajeros o aleatorios.